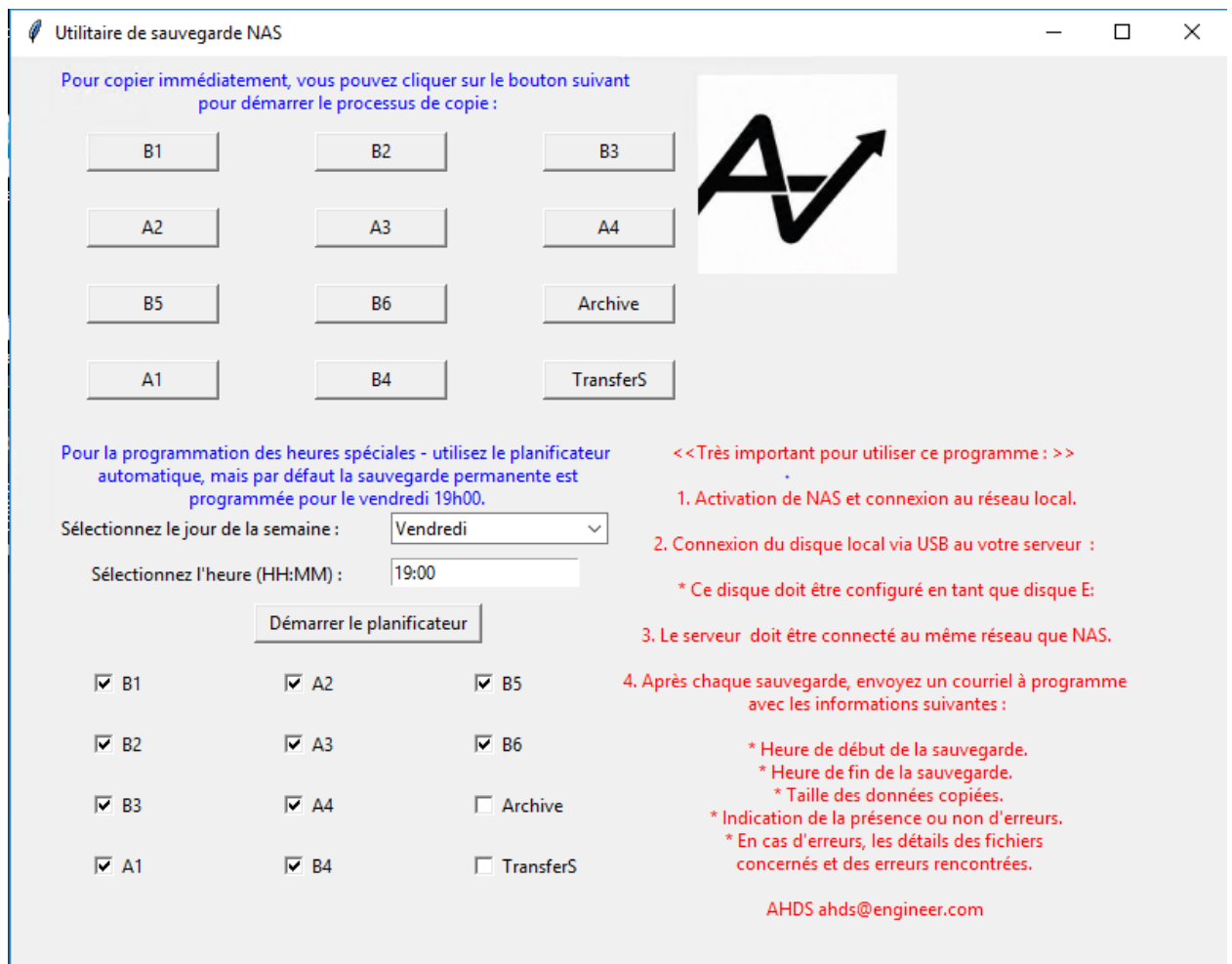


AHDS Code Library — Utilitaire de sauvegarde NAS en Python

Présentation



Cet utilitaire a été développé en **Python** afin d'automatiser la copie de données depuis un serveur **NAS** vers un disque de stockage local.

Il ne s'agit pas uniquement d'un exemple de programmation : une version de cette application est utilisée depuis **plus d'un an** pour effectuer régulièrement des sauvegardes de données.

L'application dispose d'une interface graphique développée avec **Tkinter** permettant de sélectionner les répertoires à sauvegarder, de lancer une sauvegarde immédiatement ou de programmer son exécution à un jour et une heure déterminés.

La version publiée ici a été légèrement révisée afin d'améliorer la lisibilité du code, la gestion des erreurs et le fonctionnement de la planification récurrente.

Fonctionnalités

- Interface graphique développée avec **Tkinter**
- Sélection individuelle des répertoires à sauvegarder
- Sauvegarde manuelle à partir de l'interface
- Planification hebdomadaire des sauvegardes
- Sélection du jour et de l'heure d'exécution
- Copie récursive des dossiers et sous-dossiers
- Conservation des principales métadonnées avec `shutil.copy2()`
- Exécution de certaines opérations dans des threads séparés
- Calcul de la quantité totale de données copiées
- Détection et enregistrement des erreurs
- Génération automatique d'un rapport de sauvegarde
- Envoi du rapport par e-mail via SMTP

Principe de fonctionnement

Le fonctionnement général de l'application est le suivant :

NAS → Sélection des répertoires → Copie des données → Disque local

À la fin de la sauvegarde :

Sauvegarde → Rapport → SMTP → E-mail

L'utilisateur peut également programmer une sauvegarde pour un jour et une heure déterminés.

Rapport de sauvegarde

Après chaque opération, le programme peut générer un rapport contenant notamment :

- l'heure de début de la sauvegarde ;
- l'heure de fin ;
- la taille totale des données copiées ;
- l'état de l'opération ;
- le nombre d'erreurs rencontrées ;
- les détails des erreurs, le cas échéant.

Le rapport peut ensuite être envoyé automatiquement par e-mail.

Bibliothèques utilisées

Le programme utilise principalement les modules Python suivants :

- `tkinter`
- `shutil`
- `os`
- `smtplib`
- `email.mime`
- `schedule`
- `time`
- `threading`
- `datetime`

La bibliothèque `schedule` doit être installée si elle n'est pas déjà disponible :

```
pip install schedule
```

Configuration

Avant d'utiliser le programme, adaptez les paramètres de configuration à votre environnement.

Chemins du NAS et du disque local

Exemple :

```
NAS_BASE_PATH = r'\\NAS123'  
DEST_BASE_PATH = r'E:\NAS123'
```

Remplacez ces valeurs par les chemins correspondant à votre propre installation.

Configuration SMTP

Exemple :

```
EMAIL_ADDRESS = "server@votre.ca"  
EMAIL_PASSWORD = "password"  
SMTP_SERVER = "smtp.your.com"  
SMTP_PORT = 587  
TO_EMAIL = ["toyou@votre.ca"]
```

Ces valeurs sont uniquement des **exemples**.

Utilisez les paramètres SMTP correspondant à votre propre fournisseur de messagerie.

Confidentialité et sécurité

La version publique de ce projet ne contient aucune information concernant l'environnement réel dans lequel l'application est utilisée.

Les éléments suivants ont été remplacés par des valeurs fictives ou génériques :

- nom du NAS ;
- noms des répertoires ;
- chemins réseau ;
- chemins locaux ;
- adresses e-mail ;
- paramètres SMTP ;
- informations de connexion.

Ne publiez jamais de véritables mots de passe, identifiants ou informations confidentielles dans un dépôt GitHub public.

Pour une utilisation réelle, il est recommandé de stocker les informations sensibles en dehors du code source, par exemple à l'aide de variables d'environnement.

Utilisation

1. Vérifier que le NAS est accessible sur le réseau.
2. Connecter le disque de destination.
3. Adapter les chemins dans la section de configuration.
4. Configurer les paramètres SMTP si les rapports par e-mail sont utilisés.
5. Installer la bibliothèque `schedule` si nécessaire.
6. Lancer le programme Python.
7. Sélectionner les répertoires souhaités.
8. Lancer immédiatement la sauvegarde ou utiliser le planificateur.

Exemple :

```
python bkpprog_clean_public_commente.py
```

Planification

L'interface permet de sélectionner :

- les répertoires à sauvegarder ;
- le jour de la semaine ;
- l'heure de démarrage.

La version publique du programme comprend une gestion révisée de la planification afin de permettre l'exécution récurrente des sauvegardes programmées.

Version publique

Le code publié dans ce dépôt est une **version publique et anonymisée** du programme.

La logique principale de l'application reste représentative de l'outil utilisé, mais les informations propres à l'installation d'origine ont été supprimées ou remplacées.

Le code contient également des commentaires en français afin de faciliter sa compréhension et son adaptation.

Améliorations possibles

Plusieurs améliorations peuvent être envisagées :

- utilisation de variables d'environnement pour les informations sensibles ;
 - création d'un fichier de configuration externe ;
 - journalisation des sauvegardes dans un fichier log ;
 - vérification de l'espace disponible sur le disque ;
 - comparaison des fichiers avant copie ;
 - barre de progression dans l'interface ;
 - historique des sauvegardes ;
 - amélioration de la gestion des erreurs réseau.
-

Auteur

AHDS — Code Library

Petits outils logiciels développés pour des applications pratiques d'ingénierie, de test et d'automatisation.

GitHub :

<https://github.com/Anfiska2023>

Contact :

ahds@engineer.com